

LAB 1. KIẾN TRÚC & THIẾT LẬP MÔI TRƯỜNG

Chào mừng các bạn đến với buổi học đầu tiên trong chuỗi bài xây dựng dự án SportsStore. Hôm nay, chúng ta sẽ cùng nhau đặt những viên gạch nền móng vững chắc nhất cho toàn bộ dự án: tìm hiểu về kiến trúc 3 lớp và xây dựng cấu trúc solution một cách chuyên nghiệp.

MỤC TIÊU BÀI HỌC: Sau buổi học này, bạn sẽ có khả năng:

1. **Hiểu và giải thích** được vai trò, trách nhiệm của từng thành phần trong kiến trúc 3 lớp (3-Tier).
2. **Phân biệt** rõ ràng giữa Solution và Project trong môi trường phát triển .NET.
3. **Tự tay cấu hình** một solution hoàn chỉnh với 3 project theo đúng kiến trúc đã phân tích.
4. **Thiết lập tham chiếu (dependencies)** một cách chính xác để các project có thể "giao tiếp" với nhau.
5. **Xây dựng nền tảng vững chắc**, sẵn sàng cho việc phát triển các tính năng ở những buổi tiếp theo.

YÊU CẦU KIẾN THỨC

- Kiến thức cơ bản về lập trình hướng đối tượng (OOP) với ngôn ngữ C#.
- Hiểu biết sơ lược về phát triển ứng dụng web (không bắt buộc nhưng là một lợi thế).

CÔNG CỤ CẦN THIẾT

1. **.NET 8 SDK:** Bộ công cụ phát triển phần mềm của .NET. Bạn có thể tải tại [trang chủ của .NET](#).
2. Môi trường phát triển (chọn 1 trong 2):
 - **Visual Studio 2022:** Một IDE (Môi trường phát triển tích hợp) đầy đủ và mạnh mẽ. phiên bản Community là miễn phí.
 - **Visual Studio Code:** Một trình soạn thảo mã nguồn gọn nhẹ, miễn phí và linh hoạt. Cần cài thêm extension **C# Dev Kit** của Microsoft để có trải nghiệm tốt nhất.

NỘI DUNG

Phần 1: Phân tích lý thuyết - Kiến trúc 3 lớp (N-Tier)

Trước khi bắt tay vào code, chúng ta cần hiểu rõ bản thiết kế của "ngôi nhà" mà mình sắp xây. Kiến trúc 3 lớp là một mô hình thiết kế phần mềm rất phổ biến, giúp tách biệt các mối quan tâm, làm cho ứng dụng dễ phát triển, bảo trì và mở rộng.

Hãy tưởng tượng ứng dụng của chúng ta như một nhà hàng:

1. Lớp Giao diện người dùng (Presentation Layer - SportsStore.WebUI): Đây là "khu vực phục vụ" của nhà hàng.
 - **Chức năng:** Hiển thị thông tin cho người dùng và tiếp nhận yêu cầu từ họ (ví dụ: xem danh sách sản phẩm, thêm vào giỏ hàng).
 - **Thành phần:** Controllers, Views, Models (ViewModels), các file tĩnh (CSS, JS, hình ảnh).
 - **Nó không quan tâm:** Dữ liệu lấy từ đâu, lưu trữ như thế nào. Nó chỉ biết cách "trưng bày" món ăn và nhận order.
2. Lớp Nghiệp vụ (Business Logic Layer - SportsStore.Domain): Đây là "bộ não" hay "bếp trưởng" của nhà hàng.
 - **Chức năng:** Chứa các quy tắc nghiệp vụ cốt lõi của ứng dụng. Định nghĩa các đối tượng kinh doanh là gì (ví dụ: một Product phải có Name, Price và không được có giá âm).
 - **Thành phần:** Các lớp entities (Product, Category, Order), các interfaces (ví dụ IProductRepository định nghĩa các hành vi liên quan đến sản phẩm mà không quan tâm ai sẽ thực hiện).
 - **Đặc điểm quan trọng:** Lớp này **hoàn toàn độc lập**, nó không biết đến sự tồn tại của database hay giao diện người dùng. Nó là trái tim của hệ thống.
3. Lớp Truy cập Dữ liệu (Data Access Layer - SportsStore.Infrastructure): Đây là "nhà kho" và "đầu bếp" của nhà hàng.
 - **Chức năng:** Chịu trách nhiệm giao tiếp với nguồn dữ liệu (database, file, API...). Thực thi các hành vi đã được định nghĩa ở lớp Business (ví dụ: triển khai IProductRepository để thực sự lấy dữ liệu sản phẩm từ SQL Server).
 - **Thành phần:** Các lớp Repository (ProductRepository), DbContext (nếu dùng

Entity Framework Core), các dịch vụ bên ngoài (email, thanh toán).

- **Nó biết:** Cách kết nối database, cách viết câu lệnh truy vấn, cách lấy và lưu dữ liệu.

Luồng hoạt động:

WebUI ➡ Domain ➡ Infrastructure ➡ Domain ➡ WebUI

WebUI (khách hàng order) ➡ Domain (bếp trưởng xem menu, quyết định món ăn cần gì) ➡ Infrastructure (đầu bếp lấy nguyên liệu từ kho) ➡ Domain (bếp trưởng chế biến) ➡ WebUI (món ăn được bưng ra cho khách).

Phần 2: Hướng dẫn thực hành chi tiết

Chúng ta sẽ thực hiện việc thiết lập này bằng cả 2 công cụ phổ biến là Visual Studio và Visual Studio Code.

Cách 1: Sử dụng Visual Studio 2022 (IDE)

Đây là cách tiếp cận trực quan, phù hợp với những bạn mới bắt đầu với hệ sinh thái .NET.

Tóm tắt các bước:

1. Tạo một Solution trống (Blank Solution) để chứa tất cả các project.
2. Tạo 3 Project tương ứng với 3 lớp kiến trúc: SportsStore.Domain, SportsStore.Infrastructure, và SportsStore.WebUI.
3. Thiết lập tham chiếu (Project References) để các project có thể sử dụng mã nguồn của nhau một cách hợp lý.

Hướng dẫn từng bước:

Bước 1: Tạo Solution trống

1. Mở Visual Studio 2022.
2. Chọn Create a new project.
3. Trong ô tìm kiếm template, gõ Blank Solution và chọn nó. Nhấn **Next**.
4. Đặt tên cho Solution là SportsStore và chọn vị trí lưu trữ. Nhấn **Create**.
5. Bây giờ bạn sẽ có một cửa sổ Solution Explorer trống trơn, đây chính là "thùng chứa" cho các project của chúng ta.

Bước 2: Tạo các Projects: Chúng ta sẽ lần lượt thêm 3 project vào solution này.

1. Tạo Project SportsStore.Domain (Class Library)

- Trong cửa sổ **Solution Explorer**, chuột phải vào solution SportsStore và chọn **Add > New Project....**
- Tìm kiếm template Class Library. Chọn C# Class Library. Nhấn **Next**.
- Đặt tên Project là SportsStore.Domain. Nhấn **Next**.
- Chọn Framework là **.NET 8.0**. Nhấn **Create**.
- Xóa file Class1.cs mặc định đi.

2. Tạo Project SportsStore.Infrastructure (Class Library)

- Làm tương tự như trên: Chuột phải vào solution SportsStore, chọn **Add > New Project....**
- Chọn template Class Library.
- Đặt tên Project là SportsStore.Infrastructure.
- Chọn Framework là **.NET 8.0**. Nhấn **Create**.
- Xóa file Class1.cs mặc định.

3. Tạo Project SportsStore.WebUI (ASP.NET Core Web App)

- Chuột phải vào solution SportsStore, chọn **Add > New Project....**
- Tìm kiếm template ASP.NET Core Web App (Model-View-Controller). Chọn nó. Nhấn **Next**.
- Đặt tên Project là SportsStore.WebUI. Nhấn **Next**.
- Chọn Framework là **.NET 8.0**. Các tùy chọn khác giữ nguyên. Nhấn **Create**.

Đến đây, Solution Explorer của bạn sẽ trông như thế này:

Solution 'SportsStore' (3 projects)

|-  SportsStore.Domain

|-  SportsStore.Infrastructure

|-  SportsStore.WebUI

Bước 3: Thiết lập tham chiếu (Dependencies): Đây là bước quan trọng để kết nối các

lớp lại với nhau theo đúng sơ đồ kiến trúc.

– Quy tắc tham chiếu:

- WebUI cần "thấy" nghiệp vụ và cách triển khai nó.
- Infrastructure cần "thấy" các định nghĩa nghiệp vụ để triển khai.
- Domain không "thấy" bất kỳ ai, nó là hạt nhân độc lập.

– Thực hiện:

1. WebUI → Domain & Infrastructure:

- Trong Solution Explorer, chuột phải vào project SportsStore.WebUI.
- Chọn **Add > Project Reference....**
- Trong cửa sổ mở ra, tick vào SportsStore.Domain và SportsStore.Infrastructure.
- Nhấn **OK**.

2. Infrastructure → Domain:

- Chuột phải vào project SportsStore.Infrastructure.
- Chọn **Add > Project Reference....**
- Tick vào SportsStore.Domain.
- Nhấn **OK**.

Bây giờ, solution của bạn đã được cấu trúc hoàn chỉnh và sẵn sàng để phát triển.

Cách 2: Sử dụng Visual Studio Code & .NET CLI (Command Line Interface)

Cách này sử dụng dòng lệnh, giúp bạn hiểu sâu hơn về cách .NET SDK hoạt động và rất hữu ích cho tự động hóa sau này.

Tóm tắt các bước:

1. **Tạo thư mục** cho dự án.
2. **Tạo file solution** bằng lệnh `dotnet new sln`.
3. **Tạo 3 project** bằng các lệnh `dotnet new`.
4. **Thêm các project** vào solution bằng lệnh `dotnet sln add`.
5. **Thêm các tham chiếu** giữa các project bằng lệnh `dotnet add reference`.

Hướng dẫn từng bước:

Bước 1: Chuẩn bị Terminal và Thư mục

1. Mở một terminal (Command Prompt, PowerShell, hoặc terminal tích hợp trong VS Code).
2. Tạo và di chuyển vào thư mục gốc của dự án.

```
# Tạo một thư mục mới tên là SportsStore
mkdir SportsStore

# Di chuyển vào thư mục vừa tạo
cd SportsStore
```

Bước 2: Tạo Solution và các Project

1. Tạo file Solution:

```
# Lệnh này tạo ra một file SportsStore.sln trong thư mục hiện tại
dotnet new sln
```

2. Tạo các Project:

```
# Tạo project Class Library cho lớp Domain
dotnet new classlib -n SportsStore.Domain -f net8.0

# Tạo project Class Library cho lớp Infrastructure
dotnet new classlib -n SportsStore.Infrastructure -f net8.0

# Tạo project ASP.NET Core MVC cho lớp WebUI
dotnet new mvc -n SportsStore.WebUI -f net8.0
```

– Giải thích:

- dotnet new <TEMPLATE>: Lệnh để tạo project mới từ một template.
- classlib: Template cho thư viện lớp.
- mvc: Template cho ứng dụng ASP.NET Core MVC.
- -n <NAME>: Cờ (flag) để đặt tên cho project.
- -f <FRAMEWORK>: Cờ để chỉ định phiên bản .NET Framework.

Bước 3: Thêm Project vào Solution

Lúc này, các project đã được tạo nhưng file .sln chưa biết đến chúng. Chúng ta cần thêm chúng vào.

```
# Thêm từng project vào file solution
dotnet sln add SportsStore.Domain
dotnet sln add SportsStore.Infrastructure
dotnet sln add SportsStore.WebUI
```

Bước 4: Thiết lập tham chiếu (Dependencies)

Đây là bước cuối cùng, tương tự như cách làm trên Visual Studio.

```
# 1. WebUI tham chiếu đến Domain và Infrastructure
dotnet add SportsStore.WebUI reference SportsStore.Domain
dotnet add SportsStore.WebUI reference SportsStore.Infrastructure

# 2. Infrastructure tham chiếu đến Domain
dotnet add SportsStore.Infrastructure reference SportsStore.Domain
```

– Giải thích:

- `dotnet add <PROJECT_A> reference <PROJECT_B>`: Lệnh này có nghĩa là "làm cho PROJECT_A có thể sử dụng code từ PROJECT_B".

Sau khi chạy xong các lệnh, bạn mở thư mục SportsStore bằng Visual Studio Code. Cài extension C# Dev Kit nếu bạn chưa có, và bạn sẽ thấy cấu trúc dự án hiện ra rõ ràng trong Explorer.

Phần 3: Tổng kết & Mở rộng

Kết quả

Dù bạn thực hiện theo cách nào, kết quả cuối cùng là một solution SportsStore được tổ chức chuyên nghiệp, bao gồm:

- Một file SportsStore.sln.
- Ba thư mục project: SportsStore.Domain, SportsStore.Infrastructure, SportsStore.WebUI.
- Các project đã được liên kết với nhau một cách chính xác, tuân thủ kiến trúc 3 lớp, sẵn sàng để chúng ta bắt đầu code các tính năng ở buổi học tiếp theo.

Bài tập mở rộng

Để củng cố kiến thức, hãy thử:

1. **Thêm một project mới:** Tạo một project Class Library tên là SportsStore.Tests để sau này viết unit test. Project này nên tham chiếu đến cả 3 project còn lại.
2. **Tạo lớp và interface đầu tiên:**
 - Trong SportsStore.Domain, tạo một class tên là Product.cs.
 - Trong SportsStore.Domain, tạo một interface tên là IOrderRepository.cs.
 - Trong SportsStore.Infrastructure, tạo một class tên là EfOrderRepository.cs và cho nó kế thừa (implement) từ interface IOrderRepository. (Bạn sẽ thấy Infrastructure có thể "nhìn thấy" Domain, nhưng ngược lại thì không).

TÀI LIỆU THAM KHẢO

- [1]. Tổng quan về giải pháp và dự án trong Visual Studio: Microsoft Docs - Solutions and Projects in Visual Studio
- [2]. Lệnh dotnet sln: Microsoft Docs - dotnet sln command
- [3]. Kiến trúc N-Tier trong C#: N-Tier Architecture in C# - GeeksforGeeks